

```

1 __author__ = 'Tilii: https://kaggle.com/tilii7'
2
3 import warnings
4 with warnings.catch_warnings():
5     warnings.filterwarnings('ignore')
6     import numpy as np
7     import pandas as pd
8     import matplotlib.pyplot as plt
9     %matplotlib inline
10    import matplotlib.cm as cm
11    from sklearn.decomposition import PCA
12    from sklearn.manifold import TSNE
13

```

Simply loading the files without any transformation. If you wish to manipulate the data in any way, it should be done here before doing dimensionality reduction in subsequent steps.

```

1 print('\nLoading files ...')
2 train = pd.read_csv('../input/train.csv')
3 test = pd.read_csv('../input/test.csv')
4 X = train.drop(['id', 'target'], axis=1).values
5 y = train['target'].values.astype(np.int8)
6 target_names = np.unique(y)
7 print('\nThere are %d unique target values in this dataset:' % (len(target_names)), target_names)

```



Loading files ...

There are 2 unique target values in this dataset: [0 1]

Principal Component Analysis (**PCA**) identifies the combination of components (directions in the feature space) that account for the most variance in the data.

```

1 n_comp = 20
2 # PCA
3 print('\nRunning PCA ...')
4 pca = PCA(n_components=n_comp, svd_solver='full', random_state=1001)
5 X_pca = pca.fit_transform(X)
6 print('Explained variance: %4f' % pca.explained_variance_ratio_.sum())
7
8 print('Individual variance contributions:')
9 for j in range(n_comp):
10     print(pca.explained_variance_ratio_[j])
11

```



Running PCA ...

Explained variance: 0.9944

Individual variance contributions:

0.9077327018849154
0.025423406321501626
0.011033728952072362
0.007026875356199525
0.006283416003140619
0.006119747729236367
0.005358647266808271
0.004532740198131314
0.003629307319756339
0.0029426228826495383
0.0023924601214410507
0.0017746433023195683
0.0016665349075175786
0.0015118573722779063
0.0014828537818274445
0.0012950139999157177
0.0012052148134726179
0.001072763106198477
0.0010395290634899615
0.0008415527677793451

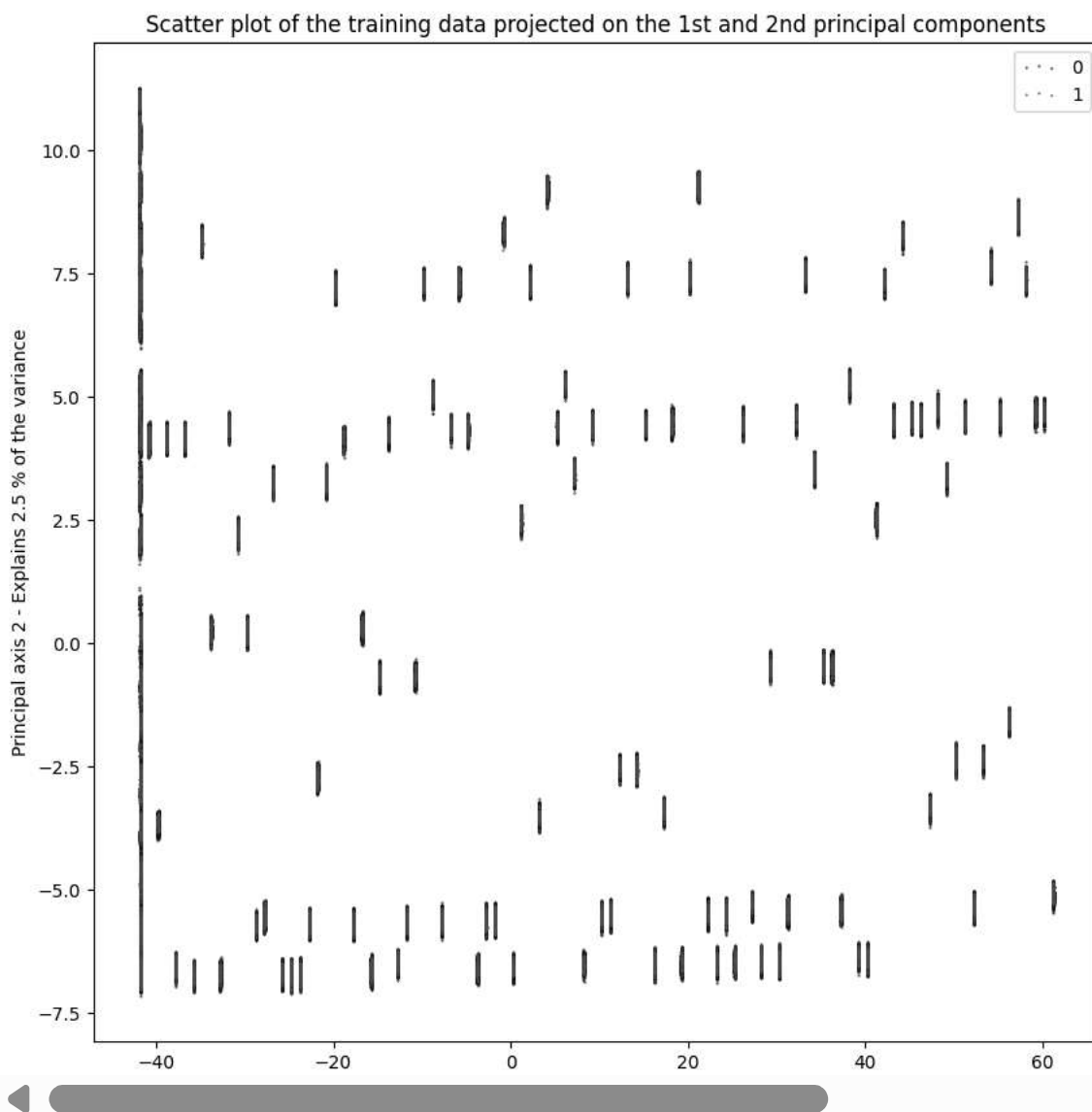
Better than 90% of the data is explained by a single principal component. Just a shade under 99% of variance is explained by 15 components, which means that this dataset can be safely reduced to ~15 features.

Here we plot our 0/1 samples on the first two principal components.

```

1 colors = ['blue', 'red']
2 plt.figure(1, figsize=(10, 10))
3
4 for color, i, target_name in zip(colors, [0, 1], target_names):
5     plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1], color=color, s=1,
6               alpha=.8, label=target_name, marker='.')
7 plt.legend(loc='best', shadow=False, scatterpoints=3)
8 plt.title(
9     "Scatter plot of the training data projected on the 1st "
10    "and 2nd principal components")
11 plt.xlabel("Principal axis 1 - Explains %.1f %% of the variance" % (
12     pca.explained_variance_ratio_[0] * 100.0))
13 plt.ylabel("Principal axis 2 - Explains %.1f %% of the variance" % (
14     pca.explained_variance_ratio_[1] * 100.0))
15
16 plt.savefig('pca-porto-01.png', dpi=150)
17 plt.show()
18

```



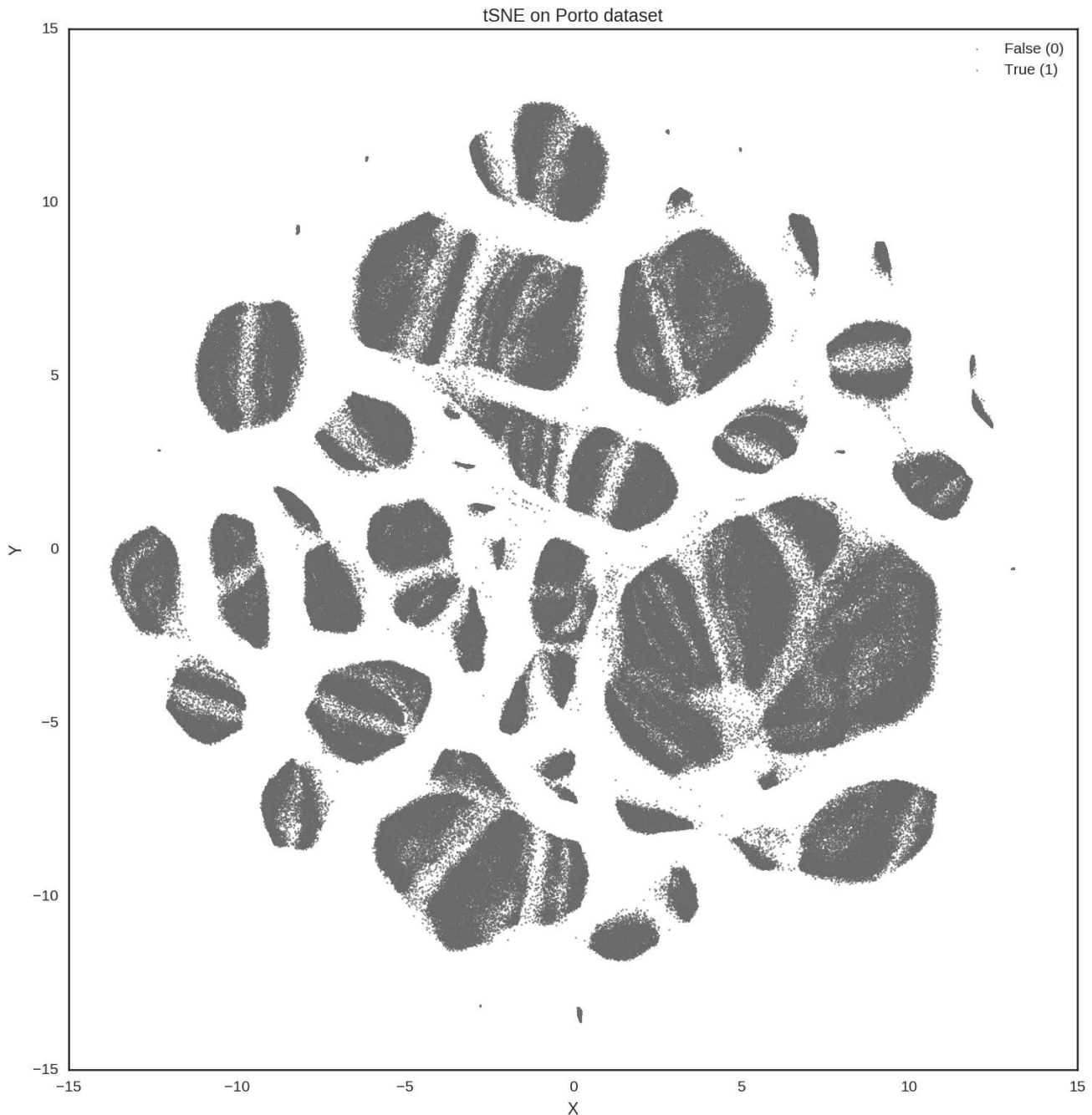
There is a nice separation between various groups of customers, but not so between 0/1 categories within each group. This is somewhat exaggerated by the fact that "0" points (blue) are plotted first and "1" points (red) are plotted last. There seems to be more red than blue in that image, even though there are >25x "0" points in reality. I'd be grateful if someone knows how to plot this in a way that would not create this misleading impression.

Regardless, 0/1 points are not separated well at all. That means that they will not be easy to classify, which we all know by now.

t-SNE could potentially lead to better data separation/visualization, because unlike PCA it preserves the local structure of data points. The problem with sklearn implementation of t-SNE is its lack of memory optimization. I am pretty sure that the t-SNE code at the very bottom will lead to memory errors on most personal computers, but I leave it commented out if anyone wants to try.

Instead, I ran t-SNE using a much faster and more memory-friendly commandline version, which can be found at the link above.

Here is the output of that exercise:



Again, we can see clear separation between different groups of customers. Some groups even have a nice "coffee bean" structure where two subgroups can be identified (gender?). Alas, there is no clear separation between 0/1 categories.

In strictly technical terms, we are screwed :D

```
1 # tsne = TSNE(n_components=2, init='pca', random_state=1001, perplexity=30, method='barnes_hut', n_iter=1000, verbose=
2 # X_tsne = tsne.fit_transform(X) # this will either fail or take a while (most likely overnight)
3
4 # plt.figure(2, figsize=(10, 10))
5
6 # for color, i, target_name in zip(colors, [0, 1], target_names):
7 #     plt.scatter(X_tsne[y == i, 0], X_tsne[y == i, 1], color=color, s=1,
8 #                 alpha=.8, label=target_name, marker='.')
9 # plt.legend(loc='best', shadow=False, scatterpoints=3)
10 # plt.title('Scatter plot of t-SNE embedding')
11 # plt.xlabel('X')
12 # plt.ylabel('Y')
13
14 # plt.savefig('t-SNE-porto-01.png', dpi=150)
15 # plt.show()
```

It was kindly brought up to me that a strange-looking PCA plot above is probably because of categorical variables in this dataset. I leave the original plot up there for posterity.

Let's encode the categorical variables and try again.

```
1 from sklearn.preprocessing import MinMaxScaler
2
3 def scale_data(X, scaler=None):
4     if not scaler:
5         scaler = MinMaxScaler(feature_range=(-1, 1))
6         scaler.fit(X)
7     X = scaler.transform(X)
8     return X, scaler
9
10 X = train.drop(['id', 'target'], axis=1)
11 test.drop(['id'], axis=1, inplace=True)
12 n_train = X.shape[0]
13 train_test = pd.concat((X, test)).reset_index(drop=True)
14 col_to_drop = X.columns[X.columns.str.endswith('_cat')]
15 col_to_dummify = X.columns[X.columns.str.endswith('_cat')].astype(str).tolist()
16
17 for col in col_to_dummify:
18     dummy = pd.get_dummies(train_test[col].astype('category'))
19     columns = dummy.columns.astype(str).tolist()
20     columns = [col + '_' + w for w in columns]
21     dummy.columns = columns
22     train_test = pd.concat((train_test, dummy), axis=1)
23
24 train_test.drop(col_to_dummify, axis=1, inplace=True)
25 train_test_scaled, scaler = scale_data(train_test)
26 X = np.array(train_test_scaled[:n_train, :])
27 test = np.array(train_test_scaled[n_train:, :])
28 print('\n Shape of processed train data:', X.shape)
29 print(' Shape of processed test data:', test.shape)
```



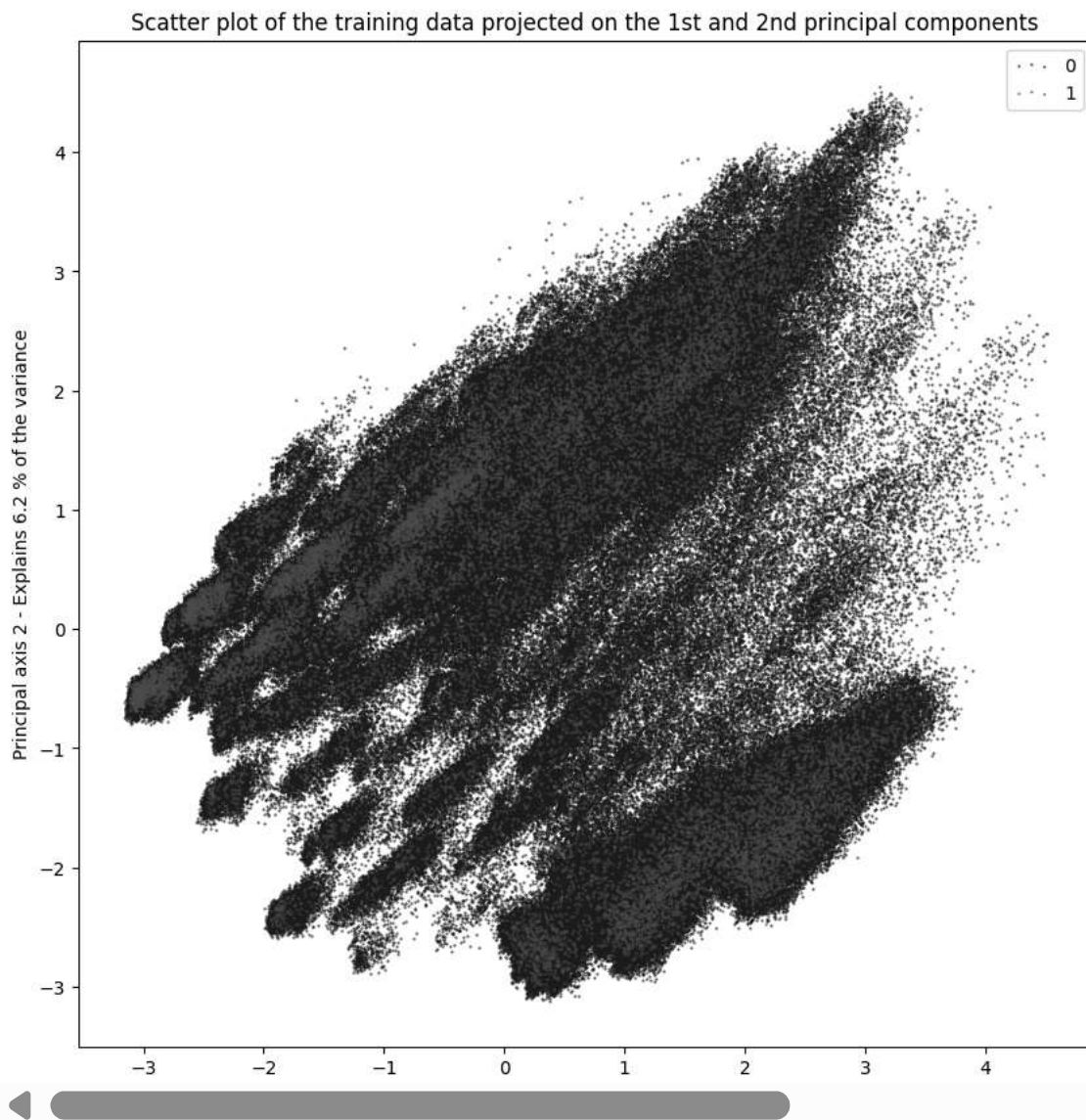
```
Shape of processed train data: (595212, 227)
Shape of processed test data: (892816, 227)
```

Repeating PCA and making another plot of the first two principal components.

```
1 print('\nRunning PCA again ...')
2 pca = PCA(n_components=n_comp, svd_solver='full', random_state=1001)
3 X_pca = pca.fit_transform(X)
4 print('Explained variance: %.4f' % pca.explained_variance_ratio_.sum())
5
6 print('Individual variance contributions:')
7 for j in range(n_comp):
8     print(pca.explained_variance_ratio_[j])
9
10 plt.figure(1, figsize=(10, 10))
11
12 for color, i, target_name in zip(colors, [0, 1], target_names):
13     plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1], color=color, s=1,
14                 alpha=.8, label=target_name, marker='.')
15 plt.legend(loc='best', shadow=False, scatterpoints=3)
16 plt.title(
17     "Scatter plot of the training data projected on the 1st "
18     "and 2nd principal components")
19 plt.xlabel("Principal axis 1 - Explains %.1f %% of the variance" % (
20     pca.explained_variance_ratio_[0] * 100.0))
21 plt.ylabel("Principal axis 2 - Explains %.1f %% of the variance" % (
22     pca.explained_variance_ratio_[1] * 100.0))
23
24 plt.savefig('pca-porto-02.png', dpi=150)
25 plt.show()
```



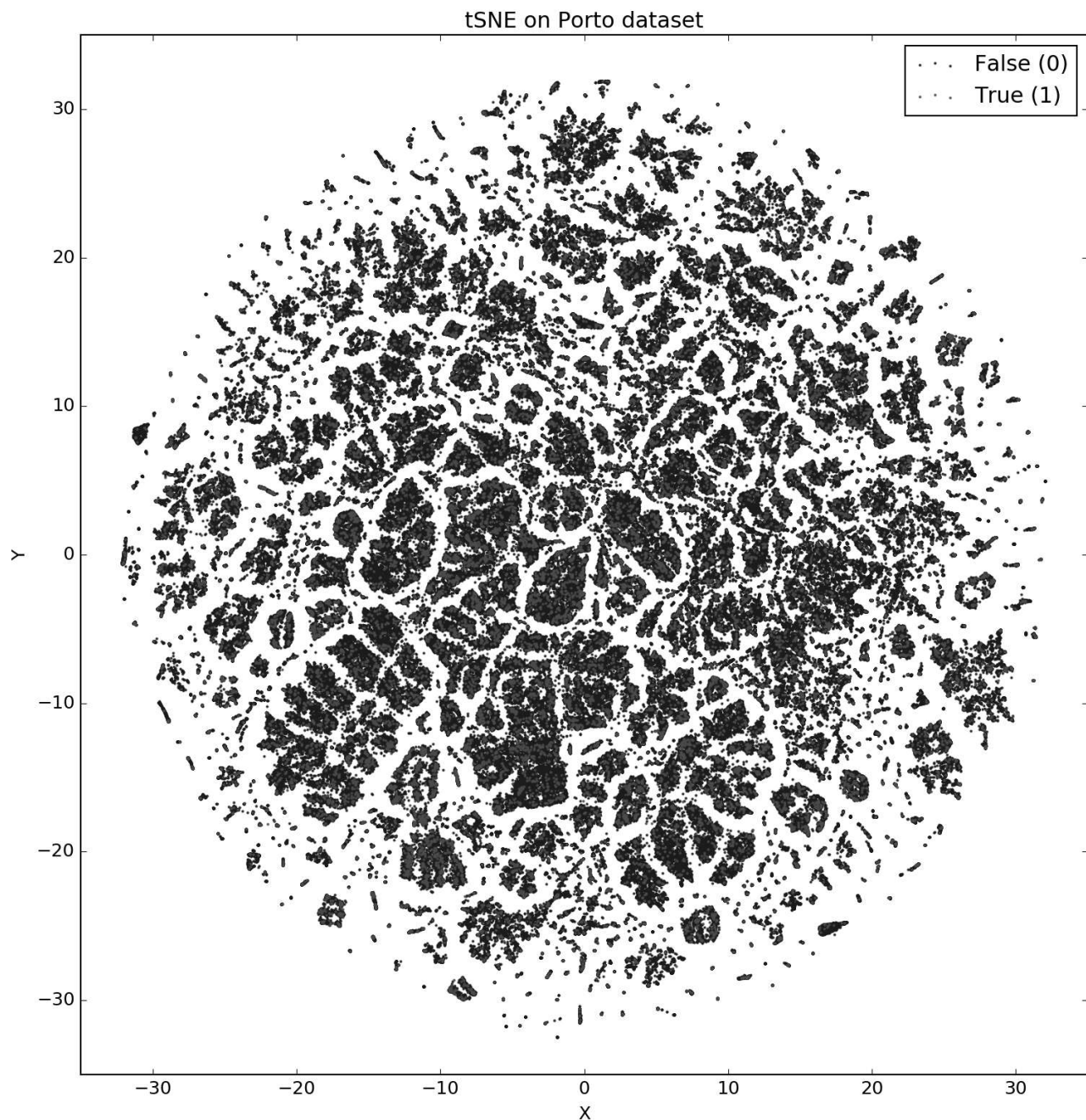
```
Running PCA again ...  
Explained variance: 0.6231  
Individual variance contributions:  
0.07408407010095992  
0.06188682036309847  
0.055953951179621216  
0.04276076114184796  
0.03590860488787132  
0.03457072503074962  
0.03146585005833527  
0.028558207829279137  
0.025510902898917925  
0.025250688029508028  
0.024582788625318087  
0.024140032207526346  
0.023864276919155773  
0.023211406602292605  
0.021035114186960507  
0.02091350289637841  
0.02006497744898973  
0.018947880007271222  
0.016250711840949422  
0.014169058538981703
```



I think that's a better plot visually and there is a good number of well-defined clusters, but still no clear separation between 0/1 points.

We can red-do the t-SNE plot as well using modified dataset. **Don't try this at home** - it takes 24+ hours using a commandline version of `bh_tsne`.

Anyway, here is the new t-SNE plot:



Again, lots of interesting clusters, but blue and red dots overlap for the most part.

This just happens to be a difficult classification problem, so maybe it is not a big surprise that raw data does not contain enough info for t-SNE to distinguish clearly between the classes.

Unfortunately, it is not much better even after training. Below is a t-SNE plot of activations from the last hidden layer (3rd) of a neural network that was trained on this dataset for 80 epochs. If you download the full version (it is roughly 10.5 x 10.5 inches), you may be able to see better that lots of red dots are concentrated in the lower left quadrant (6-9 on a clock dial), and that there are clearly fewer red dots in the upper right quadrant (0-3 on a clock dial). So the network has succeeded somewhat in sequestering the red dots, but they still overlap quite a bit with blue ones.

